

TASK 1: AMDAHL'S LAW & PHYSICAL SILICON SATURATION

Step 1

Name	Number Of Cores	Number Of LogicalProcessors
11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz	4	8

Step 2

OS Reported Logical Cores: 8

Single Process Baseline Time T(1): 3.8788s

Cores | Time(s) | Observed Speedup | Theoretical Linear

1	3.6018	1.08x	1.00x
2	2.5035	1.55x	2.00x
4	1.3769	2.82x	4.00x
8	1.2760	3.04x	8.00x
12	5.1028	0.76x	12.00x
16	1.5776	2.46x	16.00x

Step 3

Cores | Time(s) | Observed Speedup | Theoretical Linear | Parallel Efficiency[S(p)/p * 100]

1	3.6018	1.08x	1.00x	108%*
2	2.5035	1.55x	2.00x	77.5%
4	1.3769	2.82x	4.00x	70.5%
8	1.2760	3.04x	8.00x	38.0%
12	5.1028	0.76x	12.00x	6.3%
16	1.5776	2.46x	16.00x	15.4%

Step 4

Q 1.1

An inflection point is observed at $p = 8^*$, which exactly matches the number of logical cores reported by the OS (`os.cpu_count() = 8`). Up to this point, each added core provides a performance gain, albeit with diminishing returns ($108\% \rightarrow 77.5\% \rightarrow 70.5\% \rightarrow 38\%$), indicating increasing IPC and fork-latency overheads even within the range of available cores. At $p=12$, when the number of processes exceeds the machine's physical capacity ($12 > 8$ logical threads), the system is forced to multiplex more worker threads

Q 1.2

Using the empirical values $T(1) = 3.8788$ s and $T(4) = 1.3769$ s, we obtain $S(4) = 3.8788 / 1.3769 = 2.8171x$. Substituting into Amdahl's law: $2.8171 = 1/[(1-P) + P/4]$. Solving algebraically: $2.8171 \times [(1-P) + 0.25P] = 1 \rightarrow 2.8171 - 2.1128P + 0.7043P = 1 \rightarrow 2.8171 - 1.4085P = 1 \rightarrow 1.4085P = 1.8171 \rightarrow P \approx 0.860$. Thus, approximately 86% of this computation cycle is parallelizable, while the remaining ~14% (fork/spawn overhead, serialization of ``pool.map``, and result collection) constitute the strictly sequential part.

Q 1.3

When $P \approx 0.86$, the theoretical acceleration limit $S_{\text{max}} = 1/(1-0.86) = 1/0.14 \approx 7.14x$. This means that even with an infinite number of cores, this task will never be accelerated by more than ~ 7.14 times relative to single-threaded execution. If the department were to purchase a 128-core server for this specific task, 121 of the 128 cores would be useless—the performance gain would practically level off after just 8–16 cores (which is confirmed by my empirical data: the actual peak was only 3.04x at $p=8$, ch

Task 2

Step 1

Adjacent Indices (False Sharing): 1.3623s

Padded Indices (Cache-Aligned): 1.3074s

Slowdown Factor: 1.04x

Step 2

Memory Layout Configuration	Trial 1 [s]	Trial 2 [s]	Trial 3 [s]	Median Time [s]	Slowdown
Adjacent Indices (False Sharing Active) — [Baseline]					
Padded Indices (Cache-Line Aligned)					

Step 3

In CPython, the elements of a regular list are not stored as “raw” values, but as an array of 8-byte pointers (on a 64-bit system) to PyObject objects in the heap. The size of a pointer is 8 bytes.

One cache line = 64 bytes = $64 / 8 = 8$ pointers per line.

With normal (adjacent) access, indices 0, 1, 2, and 3 occupy bytes 0–31—that is, all 4 threads ($\text{index}=0..3$) fit into ONE cache line (bytes 0–63), which creates the conditions for false sharing.

With padded access, the index of thread i is calculated as $i*16$, that is, the thread

Step 4